

Mini-Projet - Le puzzle de Bix - ICC-C 2025-2026

Mini-projet d'ICC, section GM, avril 2026.

1. Instructions générales

Nous vous demandons d'écrire tout votre code dans un seul fichier nommé `puzzle.c` sur la base d'un fichier fourni, à compléter en utilisant le langage C. Nous attendons de vous un code **testé**, intelligible, correctement indenté, avec des noms de variables adéquats, et le moins de duplication (hors tests) de code possible.

Rendre un programme qui *ne compile pas* est **inacceptable** ! Cela revient à remettre une copie blanche, et recevra donc **un 0 pointé pour l'ensemble du mini-projet** ! S'il y a une fonction que vous ne parvenez pas à compiler, commentez l'intérieur jusqu'à ce que l'entièreté du programme restant compile.

- Ce mini-projet doit être réalisé **individuellement** ! L'échange de code relatif à ce projet noté et l'utilisation d'outil automatique pour le réaliser sont **strictement interdits** ! Cela inclut également la diffusion de code sur le forum du cours ou sur tout site de partage extérieur. De même, cela inclut toute intelligence artificielle.

Le code rendu doit être le résultat de *votre propre production*. Le plagiat de code, de quelque façon que ce soit et quelle qu'en soit la source, sera considéré comme fraude.

En cas de fraude, vous recevrez la note « NA » pour l'entièreté de la branche et serez de plus dénoncé-e et en subirez les conséquences suivant l'ordonnance sur la discipline.

- Vous devez donc également garder le code produit pour ce projet dans un endroit à l'accès strictement personnel.

Le fichier (source !) `puzzle.c` à fournir comme rendu de ce devoir ne pourra plus être modifié après la date et heure limite de rendu.

- Veuillez à rendre du code *anonyme*: **pas** de nom, ni de numéro SCIPER, ni aucun identifiant personnel d'aucune sorte !
- Utilisez le site Moodle du cours pour rendre votre projet.

Vous avez jusqu'au **vendredi 8 mai 2026, 23h58** (heure du site Moodle du cours faisant foi) pour soumettre votre rendu. *Aucun délai supplémentaire ne sera accordé.*

2. Introduction du sujet

Dans la série 6, Alice nous montrait ses prouesses sur un jeu vidéo avec Bix, le lapin adorable. Le niveau qu'elle nous avait montré était assez simple. Elle a maintenant atteint un niveau plus élevé, avec des énigmes plus compliquées.

Bix se trouve maintenant dans un champ avec du sol (*floor*), des blocs (*blocks*) et des trous (*holes*), ainsi qu'un goal à atteindre. Le champ est une grille de taille $W \times H$ (**w**idth et **h**eight). Chaque case du champ est d'un type parmi les suivants :

- du sol ;
- un bloc fixe ;
- un bloc déplaçable ;
- un bloc déplaçable une seule fois ;
- un trou ;
- le goal.

Le sol, le goal et les trous sont considérés comme *cases libres*. Les blocs sont considérés comme *cases occupées*. On considère aussi que l'extérieur du champ est occupé.

Bix peut se déplacer dans les 4 directions cardinales, avec les « touches » **e d s f** pour nord, sud, ouest et est, respectivement.¹ Il peut se déplacer vers une case si :

- soit la case d'arrivée est *libre* ;
- soit elle contient un bloc déplaçable (une seule ou plusieurs fois), *et* la case *derrière* ce bloc est libre.

Dans le deuxième cas, Bix *pousse* le bloc d'une case. Si le bloc se retrouve dans un trou, il tombe dans le trou et disparaît. Sinon, s'il s'agit d'un bloc déplaçable une seule fois, il se « transforme » en bloc fixe, de manière à ne plus pouvoir bouger ensuite. Dans tous les cas Bix prend l'ancienne place du bloc.

Attention ! Puisque le goal est une case libre, il est possible de pousser un bloc sur le goal. Si c'était un bloc déplaçable une seule fois, cela condamne le goal. Mais si c'est un bloc déplaçable plusieurs fois, il sera possible de le pousser plus loin et donc de révéler le goal à nouveau.

Si Bix atteint le goal, Alice remporte le niveau. S'il tombe dans un trou, alors le niveau est perdu, et *tout est réinitialisé comme au début (reset)*, sans que le programme se termine.

En plus des déplacements, Alice a encore deux commandes possibles :

- **r** (**reset**) réinitialise le jeu, comme si Bix était tombé dans un trou ;
- **x** (**exit**) quitte le jeu en abandonnant.

Cette fois, les niveaux changent. En fait, on peut même personnaliser le jeu et créer nos propres niveaux ! Alice ne connaît donc pas la séquence de touches par cœur. Elle va devoir vraiment jouer (et vous aussi !).

Pour cela, on va permettre de lire le niveau depuis un *fichier de carte*, puis de jouer au clavier. Pour l'instant, la seule façon que vous connaissez de lire un fichier *remplace* le clavier, et vous n'avez donc pas encore les outils pour faire les deux en même temps. C'est pourquoi on vous donne le code qui lit le fichier.

¹Ces lettres ont une position qui ressemble aux flèches directionnelles, que vous ayez un clavier QWERTZ, AZERTY ou QWERTY. C'est pour ça qu'on les a choisies.

3. Fichiers de carte

Un fichier de carte ressemble à ceci :

```
10 10
1 1
xxxxxxxxxx
x      ! x
x  oo   x
x      x
x  *   x
x x *   x
x x     x
x  x+++x
x      x
xxxxxxxxxx
```

La première ligne contient les dimensions W et H du champ. La seconde contient la position (x, y) où commence Bix. Ensuite viennent H lignes représentant la carte. Attention, la première ligne correspond à l'ordonnée $H - 1$. Dans cet exemple, Bix commence dans le coin *inférieur* gauche.

Les lignes contiennent *maximum* W caractères. S'il manque des caractères, il faut considérer que ce sont des espaces. Dans le fichier, les caractères ont la signification suivante :

Caractère	Signification
espace	sol
x	bloc fixe
*	bloc déplaçable
+	bloc déplaçable une seule fois
o	trou
!	goal

Pour ne pas que vous ayez à vous soucier de l'aspect « lire un fichier », nous avons défini pour vous une struct qui représente le contenu brut d'un fichier carte (*raw map*) : `rawmap_t`. Nous vous donnons deux fonctions pour les construire : une qui lit effectivement depuis un fichier (`read_map_file`) et une qui permet de les créer depuis du code (`make_rawmap`).

Dans tous les cas, c'est une représentation brute du contenu du fichier. Ce n'est pas une structure adaptée au déroulement du jeu, ne fût-ce que parce que les lignes peuvent être plus courtes que `width`. Vous devrez donc définir votre propre structure de données pour représenter votre jeu.

On vous donne aussi un petit squelette de main qui vous permettra, au choix, d'utiliser une carte par défaut écrite dans votre code, ou une carte lue depuis un fichier.

```
$ ./puzzle
```

lancera la carte par défaut, alors que

```
$ ./puzzle fichier-carte.txt
```

lancera la carte lue depuis le fichier `fichier-carte.txt` (par exemple).

4. Instructions

On vous fournit un fichier `puzzle.c` de base. Vous devez travailler dans ce fichier.

Ce fichier contient :

- la définition de `rawmap_t` est des fonctions de lecture des fichiers, que vous ne pouvez pas modifier ;
- une fonction `make_default_rawmap()`, qui crée une carte par défaut ; vous pouvez modifier cette fonction ;
- une fonction `main()` qui crée une `rawmap_t`, soit depuis `make_default_rawmap()`, soit en lisant un fichier, en fonction de si on appelle le programme avec `./puzzle` ou `./puzzle fichier-carte.txt`.

Ces préliminaires étant établis, voici les différentes tâches que nous vous demandons. Il n'y a pas nécessairement d'ordre précis dans lequel réaliser ces tâches, donc lisez totalement l'énoncé avant de commencer.

- Définir votre ou vos types de données pour représenter le jeu en cours. On l'appellera `game_t` dans les explications ci-dessous, mais vous avez totale liberté sur son nom et ce à quoi il ressemble.

Il est possible que `game_t` ressemble à `rawmap_t`. Il faut cependant qu'il soit plus facile à manipuler pour vos autres fonctions. Idéalement, faites en sorte que ces types de données ne dépendent pas des caractères précis utilisés dans les fichiers. Si on décidait de changer les `*` par des `%` dans les fichiers, cela ne devrait pas altérer votre type de données ni vos fonctions.

- Définir une fonction qui crée un `game_t` à partir d'un `rawmap_t`.
- Définir une fonction qui affiche l'état d'un `game_t` à l'écran. On affiche uniquement le contenu du champ. Bix est représenté par le caractère `@`. Les autres cases sont représentées par le même caractère que celui utilisé dans les fichiers de carte. Si un bloc se trouve sur le goal, il cache le goal ; on affiche donc `*`, pas `!`.
- Définir une fonction qui accepte (au moins) un `game_t` et une des 6 commandes possibles, et l'applique au jeu.
- Implémenter le reste : `main`, lecture des commandes au clavier, affichage et application des commandes.

À vous de structurer votre programme en définissant des fonctions supplémentaires, selon ce que vous jugez utiles.

Documentez vos fonctions et testez-les (avec des tests unitaires).

Une fois complètement implémenté, votre programme doit se comporter comme suit :

1. Lire le fichier de carte donné, ou créer la carte brute par défaut (rappel : on vous donne le code qui fait ça) ;
2. Initialiser le jeu à partir de cette carte brute ;
3. Affiche le jeu dans son état initial ;
4. Tant que Bix n'a pas atteint le goal, et qu'on n'a pas abandonné avec `x` :
 1. Lire une chaîne de caractères au clavier, qui contient une ou plusieurs commandes (par exemple, `feefrsd`) ;
 2. Pour chaque caractère de cette chaîne :
 1. Appliquer la commande ;
 2. Afficher le nouvel état du jeu ;
 3. S'arrêter si on a gagné ou abandonné.
5. Si on a gagné, afficher "Bravo ! Tu as atteint le goal !" à l'écran ; sinon afficher "Abandon :- (".
6. Terminer le programme.

4.1. Notes

- Vous pouvez jouer vous-même à votre jeu. Lancez votre programme, puis écrivez chaque commande une à une. f suivi de Enter, puis e suivi de Enter, etc.
- On vous fournit quelques cartes, ainsi que des exemples d'entrée au clavier et de sortie à l'écran attendue pour ces entrées. Votre code doit cependant pouvoir fonctionner sur n'importe quelle entrée valable.
- Vous ne pouvez pas partager de code, mais **vous pouvez partager vos fichiers de cartes**. Pourquoi ne pas défier vos collègues avec vos propres énigmes ? (N'en faites pas trop quand même ; n'oubliez pas que vous avez aussi d'autres cours.)
- Si un comportement ou une situation donnée n'est pas définie dans la consigne ci-dessus, vous êtes libre de définir le comportement adéquat. On considérera comme comportement adéquat toute solution qui ne viole pas les contraintes données et qui ne résulte pas en un crash du programme.
- N'oubliez pas de compiler et lancer votre programme avec les options de « désinfection » (*sanitize*) pour vérifier que vous ne tombez pas dans des comportements indéfinis de C. Nous testerons vos programmes avec ces options, et avoir des erreurs de ce type sera pénalisé (mais ne vaut pas 0 ; ça c'est seulement si votre programme ne compile pas).
- Nous évaluons surtout si votre code est correct (y compris le point ci-dessus). Cependant, quelques points sont dédiés à évaluer la *qualité* de votre code : est-il lisible (y compris commenté), est-il bien testé, etc.
- Comme vous ne rendez que `puzzle.c`, les fichiers de cartes que vous auriez créés en plus ne constituent pas des *tests* que nous pouvons évaluer. Seuls vos *tests unitaires* sont pris en compte pour évaluer que votre code est bien testé.